

6.S081: C 语言入门

2021 年秋季，助教：Cel Skeggs (他们/他们)

为什么使用 C?

为什么不使用 C:

- C 语言陈旧且复杂，具有微妙的行为和锋利的边缘。
- 很多最近的工作是在较新的语言中构建操作系统。Rust、Go、Java 等。

然而:

- 在操作系统工程中广泛使用。许多（并非全部）实际系统使用 C 语言编写。
- 在几乎所有平台上都受到支持。
- 迫使你更深入地理解底层机器。

C 和 Python 有什么不同？（对比 Python）

- C 大致上是一种“高级汇编语言”
 - 代码结构直接映射到实现它们的机器指令。
 - 作为对比的例子，Python 字典背后隐藏了很多底层代码。
- C 是编译的，而不是解释的。
 - 可直接在处理器上执行，无需任何底层运行时。非常快。
- C 是静态类型语言。
 - 在 Python 中，类型与变量中的值关联。
 - 在 C 中，类型与变量相关，并解释值的原始字节。
 - 类型错误在编译时被捕获。
 - 如果代码不需要检查类型，它可以执行得更快。

C 与 Python 有什么不同?

- C 使用手动内存管理，而不是垃圾回收。
 - 显式的“malloc”和“free”调用。直接访问内存。
 - 这更快，但错误更多。
- 在 C 语言中，整数和浮点数具有特定但不确定的边界。
 - 不同的类型在不同的平台上具有不同的含义。
 - 某些类型在大多数现代平台上具有共同的含义，但不是绝对的。

C vs. Python: 类型、变量和值

- 在像 Python 这样的语言中，一个值有一个类型，变量可以包含任何类型中的任何值。

```
x = 10.5
```

```
y = \"hello\"
```

```
x = y
```

- x 初始时保存一个浮点数，但随后保存一个字符串。在 Python 中这样做完全没问题。
 - 每个值都存储在内存的一个区域中，该区域包含值类型的信息。
-
- 这一点在 C 中并不适用。

C vs. Python: 类型、变量和值

- 在 C 中，值不包含任何类型信息。所有的类型信息都存储在变量中。

```
int x = 10; char *y =  
"hello, world!";
```

- 只有当一个值被存储在一个特定类型的变量中时，它才具有该类型。每个变量都对应一个足够大的内存区域来存储该值。
- 这种类型信息只存在于程序编译时。程序运行时并不存在这种类型信息。

C 中的原始整数类型

Type	Size on 64-bit RISC-V	有符号最小值/最大值 在 64 位 RISC-V 上	无符号最小/最大 在 64 位 RISC-V	保证最小值 跨平台
[有符号/无符号] char	1 字节整数	-128 到+127	0 到 255	8 位, -127 到+127
[无符号] short	2 字节整数	-32768 到+32767	0 到 65535	16 位, -32767 到+32767
[无符号] int	4 字节整数	-2^31 到 2^31 - 1	0 到 2^32 - 1	16 位, -32767 到 +32767
[无符号] long	8 字节整数	-2^63 到 2^63 - 1	0 到 2^64 - 1	32 位, -(2^31-1) 到 2^31-1
[unsigned] long long	8 字节整数	-2^63 到 2^63 - 1	0 到 2^64 - 1	64 位, -(2^63-1) 到 2^63-1

在这个类中你不需要担心保证的最小值.....当你转向为其他平台编写 C 代码时，只需记住这一点即可。

C 中的基本浮点类型

类型大小在 RISC-V 格式在 RISC-V 保证通常			
float 4 字节浮点数 32 位 IEEE 754-2008 None			
double 8 字节浮点数 64 位 IEEE 754-2008 至少和 float 一样长			
long double 16 字节浮点数 128 位 IEEE 754-2008 至少和 double 一样长			

在 C 中定义原始变量

```
int value_1;  
int value_2 = 83; float value_3 = 125.0;  
char value_4, value_5 = 3, value_6 = 0xFF;
```

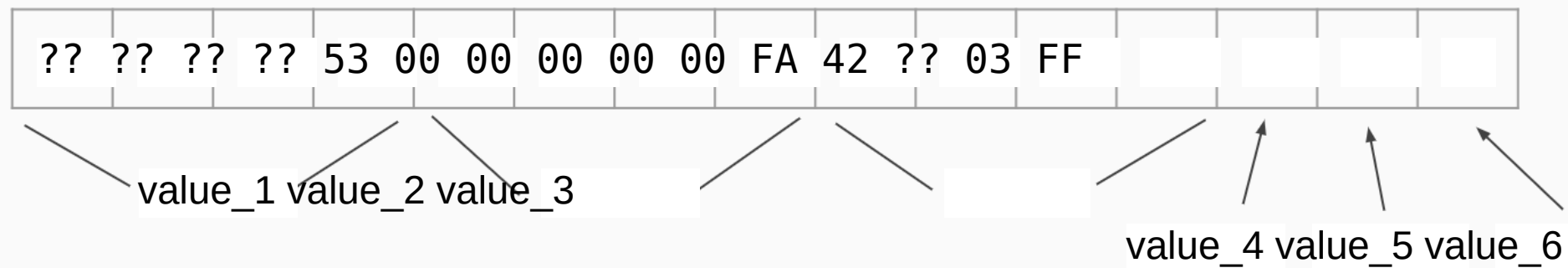
变量定义包含三部分：类型、名称和（可选地）初始化器。

可以一次定义多个变量，但这可能会变得混淆。

如果没有初始化一个变量，它可能会默认为零，但你不能依赖这一点.....你必须在使用前初始化它！（除了静态/全局变量。）

原始变量的内存

```
int value_1;  
int value_2 = 83; float value_3 = 125.0;  
char value_4, value_5 = 3, value_6 = 0xFF;
```



(这些变量在内存中的可能布局.....不保证!)

字节序

我们如何在内存中写入一个像 0x12345678 (= 305419896) 这样的数字？

0x12, 0x34, 0x56, 0x78 - 或 - 0x78, 0x56, 0x34, 0x12
(大端序) (小端序)

取决于平台！这是一个长期的技术争论。我们特定的 RISC-V 平台是小端序，所以 6.S081 也是小端序的。

助记符：大端序意味着从大端开始（即 MSB，或“最显著的字节”）。

额外知识：阅读 IEN 137（1980 年）以了解术语的来源（格列佛游记）。

C 中的内存类型

- 堆栈内存

- 函数内部分配的局部变量。此内存会在函数退出后销毁并可能被重用。
- 默认未初始化。将反映该内存区域原本的状态。

- 静态内存

- 在任何函数外部声明的变量，以及使用“static”声明的变量。
- 仅存储一份，在预定义且不变的内存地址中。
- 默认初始化为零。

- 堆内存

- 显式分配（malloc）和释放（free）。
- 释放后，内存可能会被重新使用（全部或部分）。○ 默认不会初始化。会反映该内存区域之前的状态。

C 中的关键主题：内存安全性

运行 C 程序的方式有很多可能会出错。这些可能会导致神秘且令人困惑的错误。一些示例：

- 使用后释放：如果程序释放了一块内存，但仍然继续使用它。
- 双重释放：如果程序两次而不是一次释放内存区域。
- 未初始化的内存：如果程序使用了从未初始化的内存。
- 缓冲区溢出：如果程序修改了内存区域之外的数据。
- 内存泄漏：如果程序分配了内存，但从未释放它。
- 类型混淆：如果程序无意中使用错误的数据类型来访问内存中的变量。

C 中的关键主题：指针

在 C 中，我们使用指针来表示内存区域：

```
int value_1 = 6828; int *pointer_to_value_1 =  
&value_1; *pointer_to_value_1 = 6081; printf("%d\n",  
value_1); // 打印的是 6081，而不是 6828!
```

指针是引用，描述底层内存位置。（内存的起始位置，但不是大小。）

指针实际上是隐藏的整数

```
int global_variable = 0; void  
test_function(void) { int local_variable = 0;  
int *heap_reference = malloc(sizeof(int));  
printf(\"Global: 0x%x\\n\", &global_variable);  
printf(\" Local: 0x%x\\n\", &local_variable);  
printf(\" Heap: 0x%x\\n\", heap_reference); }
```



```
$ test  
全局: 0x8C8  
Local: 0x2FAC  
Heap: 0x12FF0
```

指针是整数，指定内存区域的起始地址，并预期找到该地址处的值类型。

各种不同的指针

```
int *a; // 指向 int 的指针 float *b; // 指向 float 的指针 int **c; // 指向  
指向 int 的指针 char (*d)(int); // 指向函数的指针 (int -> char) char (**e)  
(int); // 指向指向函数的指针 (int -> char) void *f; // 未类型化的内存指针  
void **g; // 指向未类型化内存的指针
```

指针可以任意嵌套：

```
int *****value; // 请不要真的这样做。
```


指针示例

```
int a = 10;  
int *b = &x;  
int c = x;  
int d = x;  
*b = 20;  
d = 30;
```

// 这会打印什么?

```
printf(\"a = %d, *b = %d, c = %d, d = %d\\n\", a, *b, c, d);;
```

指针示例

```
c int a = 10;  
int *b = &x;  
int c = x;  
int d = x;  
*b = 20;  
d = 30;
```

// 这个打印什么?

```
printf(\"a = %d, *b = %d, c = %d, d = %d\\n\", a, *b, c, d);;
```

答案: a = 20, *b = 20, c = 10, d = 30

另一种数据类型：数组

```
int fibonacci_numbers[] = {1, 1, 2, 3, 5, 8, 13, 21};  
int even_numbers[6] = {0, 2, 4, 6, 8, 10}; int  
uninitialized_array[6]; int my_array[6] = {1};
```

```
my_array[3] = 100;  
my_array[4] = 200; printf("my_array: [0] = %d, [3] =  
%d, [5] = %d\n",  
    my_array[0], my_array[3], my_array[5]);
```

另一种数据类型：数组

```
int fibonacci_numbers[] = {1, 1, 2, 3, 5, 8, 13, 21};  
int even_numbers[6] = {0, 2, 4, 6, 8, 10}; int  
uninitialized_array[6]; int my_array[6] = {1};
```

```
my_array[3] = 100;  
my_array[4] = 200; printf("my_array: [0] = %d, [3] =  
%d, [5] = %d\n",  
    my_array[0], my_array[3], my_array[5]);
```

答案: [0] = 1, [3] = 100, [5] = 0. (是的，部分初始化数组的其他元素也会被初始化!)

数组说明

- 数组不是列表。数组具有固定长度，并且不可调整大小！
- 数组元素按顺序排列在内存中：

```
int my_array[4]; printf(\"位置： %x %x %x %x\\n\",  
&my_array[0],  
&my_array[1], &my_array[2], &my_array[3]); //  
打印结果：地址： 2FB0 2FB4 2FB8 2FBC
```

- 请注意，数组是基于 0 索引的，而不是 1 索引。与 Python 不同，你不能使用负索引来从末尾倒数。

指针算术

- 引用数组的第 n 个元素 ($\text{array}[n]$) 与引用数组指针加上 n 之后的内存内容 ($\text{array} + n$) 是相同的
- 取数组的第 n 个元素的引用 ($\&\text{array}[n]$) 与将 n 加到数组指针 ($\text{array} + n$) 是相同的。
- 但等等..... $\&\text{my_array}[3]$ 是 $0x2FBC$ ，比 $0x2FB0$ 大 12!
- 这是因为指针算术会乘以底层数据类型的大小!

```
(long) (my_array + 3) == ((long) my_array) + 3 * sizeof(int)
```

指针算术的一个副作用

这会打印什么？

```
int values[5] = {10, 20, 30, 40, 50};  
printf("%d\n", 4[values]);
```

(考虑...如果 $x[y] == x + y$ ，且我们知道加法是可交换的...)

Answer: 50! $x[y] = *(x+y) = *(y+x) = y[x]$

数组的大小

```
"int a[10]; printf(\"sizeof(a)=%d, sizeof(a[0])=%d,  
num_elems=%d\\n\", sizeof(a), sizeof(a[0]), num_elems);\",  
    sizeof(a), sizeof(a[0]), sizeof(a)/sizeof(a[0]));
```

```
// 输出: sizeof(a)=40, sizeof(a[0])=4, num_elems=10
```


原始类型之间的类型转换

```
float a = 100; // 赋值一个整数给浮点数是没有问题的! int b = a; //  
但是将浮点数赋值给整数就不行! // 为了解决这个问题...我们需要进行类型转  
换! float c = 701.7; int d = (int) c; // 接收值 701! (截断。)
```

指针和整数之间的类型转换

```
// 将指针转换为整数 int x[4] = {1, 2, 3, 4}; int *x_ptr = &x[0]; long x_address = (long) x_ptr;
// 从整数转换为指针 int *x2_ptr = (int *) (x_address + 4); int x2_value = x2_ptr[1];
```

// x2 value 的值是多少?

指针和整数之间的类型转换

```
// 将指针转换为整数 int x[4] = {1, 2, 3, 4}; int *x_ptr = &x[0]; long x_address = (long) x_ptr;  
// 将整数转换为指针 int *x2_ptr = (int *) (x_address + 4); int x2_value = x2_ptr[1];
```

```
// x2_value 有什么值?
```

Answer: x2_value 是 3!

Casting: 一个可能导致内存不安全的问题

```
long x = 0xDEADBEEF;  
int *x2 = (int *) x;  
*x2 = 12345;  
printf(\"Done!\\n\");
```

发生了什么?

不同平台会给出不同的错误。在 Linux 上，你可能会得到一个段错误。这是 xv6 的错误消息。

```
$ test usertrap(): unexpected scause 0x0000000000000000f  
pid=3  
sepc=0x000000000000000016 stval=0x00000000deadbeef  
$
```

指针的大小是多少？

- 取决于具体的平台。
- 我们的 RISC-V 变体使用 64 位（8 字节）指针，这使得它们与 long 的大小相同。
- (通常但不总是，指针与 long 的大小相同。)
- 这意味着在 6.S081 中，我们可以将指针转换为 long，但不能转换为 int。类型为 'int' 的变量太小，无法容纳完整的指针。

C 中的函数

```
double add_numbers(double *numbers, int count) {  
    // ^ numbers 和 count 在栈上  
    double result = 0; // <- result 也在栈上  
    for (int i = 0; i < count; i++) { // <- 'i' 也在栈上  
        result += numbers[i]; }  
    // 返回后，所有栈变量都会消失!  
    return result; }
```

一种不寻常的数据类型：void

- 表示没有数据类型。
- 主要在函数的返回类型和参数中使用：

```
void test_function(void);
```

- 你不能定义一个 void 类型的变量，因为.....那是什么意思？
- 但你可以定义一个 void * 类型的变量。你只是不能解引用它。
- (官方不允许在 void * 指针上进行算术运算，但我们使用 GCC，它支持这是 C 语言的一个扩展。)

Oops

官方不允许在 void * 指针上进行算术运算，但我们使用 GCC，它支持这是 C 语言的一个扩展。这段代码有什么问题？

```
int *multiples_of(int number, int max) {  
    int my_local_array[max]; for (int i = 0; i  
    < max; i++) {  
  
        my_local_array[i] = number * (i + 1); }  
    return &my_local_array[0]; }
```


Oops

这段代码有什么问题？

```
int *multiples_of(int number, int max) {  
    int my_local_array[max];  
    for (int i = 0; i < max; i++) {  
        my_local_array[i] = number * (i + 1);  
    }  
    return &my_local_array[0];  
}
```

Answer: my_local_array 将在函数返回时不再存在，因此返回的指针将无效，并且很可能立即被垃圾数据覆盖！如果该指针被修改，情况会更糟！

Oops (fixed)

解决方案:

```
int *multiples_of(int number, int max) { int  
*my_local_array = malloc(sizeof(int) * max); for (int  
i = 0; i < max; i++) {  
my_local_array[i] = number * (i + 1); } return  
&my_local_array[0]; } // 调用者需要调用 free() 释放返回的指针。
```

定义与声明

```
extern void function_1(void); // 声明一个函数  
extern int some_variable; // 声明一个变量
```

```
void function_1(void); // 也声明一个函数  
int some_variable; // 但这是定义一个变量!
```

```
void function_2(void) { // 这定义了一个函数! // ... }
```

定义与声明

- 在使用之前，您必须在每个文件中声明一个变量或函数，因为 C 需要知道其类型或类型签名。
 - 您可以多次声明一个变量或函数，只要它始终具有相同的类型或类型签名即可。
- 您必须在代码库中精确地定义每个变量或函数一次。
 - 一个定义也作为一个声明，但（当然）只有在定义发生之后。
 - 你可以在一个文件中定义一个函数或变量，但在另一个文件中使用它。
- 我们通常将许多声明放在单独的“头文件”中，以便程序的每一部分都知道其他部分的重要类型。
 - 诸如 `#include "kernel/types.h"` 的语句用于引入头文件。

需要声明的一个例子

```
void function_1(void) { function_2(100); // 这里无法工作! }
```

```
void function_2(int xyz) {  
    printf("%d\n", xyz); }
```

C 从上到下执行。如果它还没有看到一个声明，它就不会知道要使用的正确类型。它如何知道 function_2 接受一个整数，而不是（例如）一个浮点数？

修复最后一个示例

```
void function_2(int xyz);
```

```
void function_1(void) { function_2(100); // 这  
现在可以工作了! }
```

```
void function_2(int xyz) {  
printf("%d\n", xyz); }
```

声明静态函数和变量

- 如果我们在一个单独的文件中将一个函数命名为相同的名称，它们将会冲突！C 会难以区分它们。
- 为了避免冲突，我们可以将我们的变量和值声明为静态：

```
static void function_2(int xyz);
```

```
static void function_2(int xyz) {  
    printf("%d\\n\\", xyz); }  
  

```

- 此函数仅可在定义它的文件中访问！

声明局部变量为静态

- 在函数内部，通常局部变量会在堆栈上分配，但我们也可以指定它们在静态内存中分配：

```
int add_cumulative_numbers(int increase) {  
    static int total_sum = 0; total_sum +=  
    increase; return total_sum; }
```

- `total_sum` 在程序启动时会被初始化为零，并且在调用 `add_cumulative_numbers` 时会保持其值！它不会被重新初始化。

函数指针

```
static void my_function_1(int);  
static void my_function_2(int);  
  
void pointer_example(int variant) {  
    "void (*local)(int);",  
    if (variant == 1) {  
        local = my_function_1; }  
    else {  
        local = my_function_2; }  
    local(100); // call  
    function via variable }  
  
  

```

C 语言字符串

- 字符串就是字符数组。

```
char *my_string = "6.S081!"; // use " for string literals
printf("string=\"%s\", third_char='%c'\n",
my_string, my_string[2]); // 打印结果:
string="6.S081!", third_char='S'
```

C 中的字符

- C 中的字符只是整数，在我们的情况下是 1 字节长。
 - 数字和字母之间的映射由 ASCII 定义，您可以在维基百科上阅读相关介绍：
<https://zh.wikipedia.org/wiki/ASCII>

```
third_char='S' char c1 = 'a'; // 使用 ' 表示字符字面量
char c2 = c1 + 1; printf("%c=%d, %c=%d\n", c1, c1,
c2, c2); // 输出: a=97, b=98
```

- 有时 'char' 用来表示字符串中的一个字符.....有时它仅仅用来表示一个字节。C 语言不区分这两种用法。

字符串的长度

- 任何 C 字符串的最后一个字符是'\0' (= 0x00)，并且 C 使用它来确定字符串的长度。
- 分配字符串时请确保留出空间以容纳这个终止符！
- 这表示你可以像下面这样计算字符串的长度：

```
int strlen(const char *str) { int i;  
for (i = 0; str[i] != 0; i++) {}  
return i; }
```

- (xv6 为你提供了一个 strlen 的实现。)

类型定义

- 不满意内置类型？你可以定义自己的！

```
// 从 kernel/types.h:  
typedef unsigned char uint8;  
typedef unsigned short uint16;  
typedef unsigned int uint32;  
typedef unsigned long uint64;
```

- 如果将来我们将 xv6 迁移到另一个平台，而该平台的大小不符合这种匹配方式，我们只需更改定义这些 typedef 的一个文件，其余的代码将自动更新以匹配！

头文件 (.h) 和源文件 (.c)

在 C 语言中，我们将包含共享声明的头文件与包含不同代码部分定义的源文件分开。

kernel/spinlock.h: 描述自旋锁接口的声明 kernel/spinlock.c: 实际的自旋锁代码定义

要在.c 文件中包含一个.h 文件：

```
#include "spinlock.h"
```

如果头文件在不同的目录中，您可能需要更长的路径，例如“kernel/spinlock.h”。另外，不要包含.c 文件！这会导致问题。

C 预处理器

C 代码中以 '#' 开头的行是预处理器指令。

一些基本示例：

`#include "spinlock.h"` -> 在此处包含 `spinlock.h` 的内容
`#define NPROC 64` -> 将所有实例的 '`NPROC`' 替换为 '`64`'
`#define TWICE(x) ((x)*2)` -> 将任何表达式 '`x`' 的所有实例替换为 '`((x) * 2)`' ...

预处理器完全在任何实际编译之前执行。在 `xv6` 中我们不经常使用它，但最终值得理解。

更多的预处理器指令

```
#ifdef DEBUG // 只有在使用 #define 定义了 DEBUG 时  
printf("一些调试信息: %d", my_value); #else  
  
// 而不是打印, 执行其他操作 #endif
```

此外, 除了 `#ifdef`, 还有 `#ifndef VAR` 如果 `VAR` 没有通过 `#define` 定义 `#if` `EXPR` 如果 `EXPR` 评估为真 还可以使用 `#undef VAR` 来取消定义通过 `#define VAR` 定义的宏

#include 保护宏

如果一个 .h 文件被包含两次，它的内容将会被复制粘贴两次。有时候这并不理想.....在这种情况下，通常会使用 include 保护：

```
// 在 something.h 文件的开头 #ifndef  
SOMETHING_H #define SOMETHING_H
```

```
// ... 这里是正常内容 ... #endif /*  
SOMETHING_H */
```

(实际上，xv6 并不真正使用此功能。)

注释的语法

// 这是一个注释，将持续到本行结束

/* 这也是一个注释，
但可以跨多行扩展，并不会在遇到终止符： */ 之前停止

常见的代码结构：if

```
if (X) { /* 只要在 X 非零，即为真时运行这段代码 */ } else if  
(Y) { /* 如果 X 不为真，但 Y 为真时运行这段代码 */ } else { /*  
如果 X 和 Y 都不为真，则运行这段代码 */ }
```

常见的代码结构：while

```
while (COND) { // 如果 COND 为真，运行这里的代  
码。
```

```
// 这段代码结束后，再次检查 COND。} // 只有当循环结束时 COND 为假时我  
们才会停止
```

常见的代码结构：do while

```
do { // 至少运行此代码一次 // 但在循环结束时我们将检查 COND // 如果  
    为真，我们将再次执行此操作 } while (COND); // 只有在循环结束时 COND  
    为假时我们才会停止
```

常见的代码结构：for

```
for (A; B; C) {  
    // BODY  
}
```

这相当于：

```
A;  
while (B) {  
    // BODY  
C;  
}
```

for 循环的示例

```
void print_even_numbers(int max) { for (int  
i = 0; i < max; i = i + 2) {  
    printf("%d\\n\\n", i);  
}  
}
```

一些方便的语法你可以使用：

`i += 2` -> 这等价于 `i = i + 2` `i++` -> 这等价于 `i = i + 1`

你可能会用到的一些常见内存函数

- `malloc(n)`: 从堆内存中分配 `n` 字节的空间，并返回指向该空间起始位置的指针。如果没有足够的内存进行分配，则返回 `NULL`。
- `free(ptr)`: 释放由 `malloc` 分配的 `ptr` 开始的内存区域。如果 `ptr` 为 `NULL`，则不执行任何操作。
- `memset(ptr, v, n)`: 将 `ptr[0]` 到 `ptr[n-1]` 中的每个字节设置为 `v`。
- `memmove(dst, src, n)`: 将 `src[0]...src[n-1]` 复制到 `dst[0]...dst[n-1]`
- `memcpy(dst, src, n)`: 一种更快的替代 `memmove` 的版本，但如果 `dst` 和 `src` 以任何方式重叠，则可能会导致错误行为。（不推荐！请使用 `memmove`。）

常用的字符串函数

- `strlen(str)`: 计算并返回字符串 `str` 的长度，基于找到其空终止符。如果空终止符缺失，将导致错误行为！
- `strcmp(a, b)`: 比较两个字符串 `a` 和 `b`，并根据 `a < b`、`a == b` 或 `a > b` 返回一个整数 `< 0`、`== 0` 或 `> 0`。
- `strcpy(dst, src)`: 等同于 `memcpy(dst, src, strlen(src)+1)`;

始终记得空终止符！

更多数据类型：结构体

- 结构让你可以声明一组不同类型的值作为一个单一的组合值。

```
struct xy_point {  
    double x;  
    double y;  
}; struct xy_point my_point = { 12.5, -6.2 };  
my_point.x = 50.6; // 修改字段‘x’ printf(\"%f\\n\",  
my_point.y); // 读取字段‘y’
```

复杂的结构

您也可以通过字段名称而不是字段在结构中的顺序来初始化结构字段：

```
struct xy_point my_point = {  
    .y = -6.2, .x = 12.5, };
```

您也可以给结构体命名，而不需要使用 'struct' 这个词：

```
typedef struct xy_point xy_point_t;  
xy_point_t my_point;
```

像结构一样，但更差：联合体

- 结构类似于联合，只是每个字段都存储在相同的内存地址：

```
union my_union {  
    float x;  
    int y;  
}
```

- 这意味着你只能在同一时间安全地使用联合中的单个字段.....并且你将需要其他方式来跟踪哪些字段是安全的。
- 你不会经常需要这些。如果你需要详细信息，可以查找它们。

逻辑运算符

(请注意: true 代表任何非 0 值, false 代表 0。)

```
int boolean_1 = 1, boolean_2 = 0;
```

```
(boolean_1 && boolean_2) == 0 // 逻辑与 (boolean_1  
|| boolean_2) == 1 // 逻辑或 !boolean_2 == 1 // 逻辑  
非
```

位运算符

```
boolean_2 == 1 // 逻辑非 unsigned short a = 0x1313, b = 0x3232;
```

```
(a & b) == 0x1212 // 位与 (a | b) == 0x3333  
// 位或 (a ^ b) == 0x2121 // 位异或 ~a ==  
0xECEC // 位非
```

下一个.....关于指针的挑战!

应用位运算符

```
unsigned int my_int; // 设置整数的第 N 位: my_int |= 1 <<
N; // 清除整数的第 N 位: my_int &= ~(1 << N); // 检查 MASK
中的任何位是否设置 if (my_int & MASK) { /* ... */ } // 检查
MASK 中的所有位是否设置 if ((my_int & MASK) == MASK) { /*
... */ } // 检查整数是否是 2 的幂 if (my_int && !(my_int &
(my_int - 1))) { /* ... */ }
```

最终：一个指针挑战

来源：[见下一张幻灯片]

```
int main() { int x[5]; // x 的地址是 0x7ffdfbf7f00
printf("%p\n", x); printf("%p\n", x+1);
printf("%p\n", &x); printf("%p\n", &x+1); return 0; }
```


最后：一个指针挑战：SOLUTION

Source: The Ksplice Pointer Challenge - Oracle Linux Blog

```
int main() { int x[5]; // x 是 0x7ffdfbf7f00
printf("%p\n", x); // -> 0x7ffdfbf7f00
printf("%p\n", x+1); // -> 0x7ffdfbf7f04
printf("%p\n", &x); // -> 0x7ffdfbf7f00
printf("%p\n", &x+1); // -> 0x7ffdfbf7f14 return 0;
}
```

有问题吗？